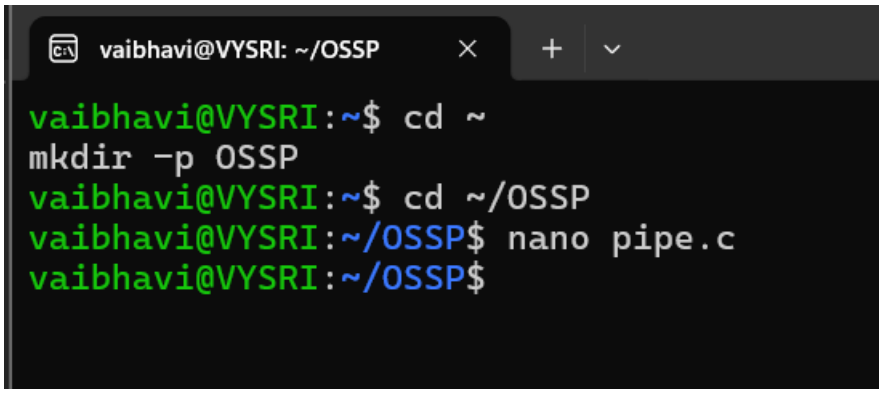
Name: Vaibhava Sree

Roll No: 2520030398

Practical 5:

1. Develop a program that executes the equivalent of the shell command: ls -l | grep ".c" using fork(), pipe(), dup2(), and exec() system calls.



vaibhavi@VYSRI: ~/OSSP

GNU nano 8.7.1 pipe.c *

```c
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main()
{
    int fd[2];

    pipe(fd);

    if(fork() == 0)
    {
        dup2(fd[1], STDOUT_FILENO);

        close(fd[0]);
        close(fd[1]);

        execlp("ls", "ls", "-l", NULL);
    }

    if(fork() == 0)
    {
        dup2(fd[0], STDIN_FILENO);

        close(fd[0]);
        close(fd[1]);

        execlp("grep", "grep", ".c", NULL);
    }

    close(fd[0]);
    close(fd[1]);

    wait(NULL);
    wait(NULL);

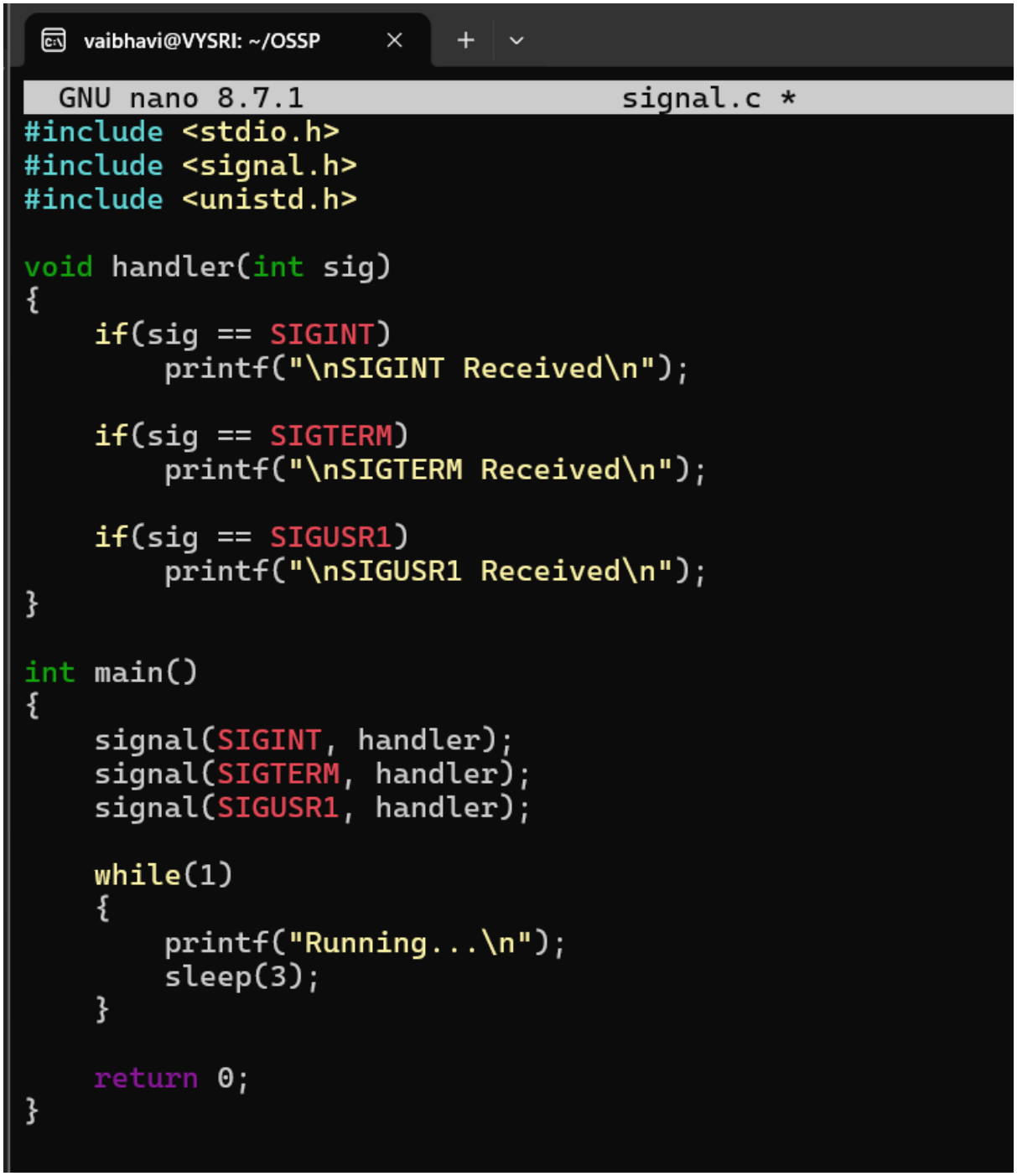
    return 0;
}
```

OUTPUT:

```
vaibhavi@VYSRI:~/OSSP$ gcc pipe.c -o pipe
vaibhavi@VYSRI:~/OSSP$ ./pipe
drwxr-xr-x 2 vaibhavi vaibhavi  4096 Sep 29 14:47 include
-rwxr-xr-x 1 vaibhavi vaibhavi 16208 Oct 13 03:38 pipe
-rw-r--r-- 1 vaibhavi vaibhavi   505 Oct 13 03:35 pipe.c
drwxr-xr-x 2 vaibhavi vaibhavi  4096 Sep 29 14:50 src
vaibhavi@VYSRI:~/OSSP$
```

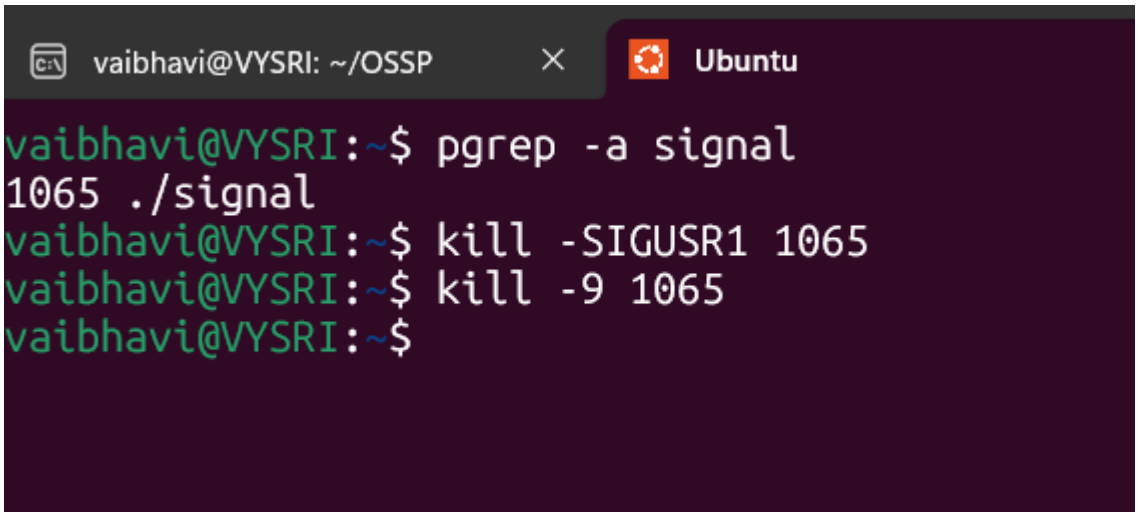
2. Create a POSIX signal-handling program that captures SIGINT, SIGTERM, and SIGUSR1. Demonstrate asynchronous event handling and explain the role of signal handlers.

```
vaibhavi@VYSRI:~/OSSP$ cd ~/OSSP
vaibhavi@VYSRI:~/OSSP$ nano signal.c
vaibhavi@VYSRI:~/OSSP$
```


vaibhavi@VYSRI: ~/OSSP
GNU nano 8.7.1 signal.c *
#include <stdio.h>
#include <signal.h>
#include <unistd.h>
void handler(int sig)
{
if(sig == SIGINT)
printf("\nSIGINT Received\n");
if(sig == SIGTERM)
printf("\nSIGTERM Received\n");
if(sig == SIGUSR1)
printf("\nSIGUSR1 Received\n");
}
int main()
{
signal(SIGINT, handler);
signal(SIGTERM, handler);
signal(SIGUSR1, handler);
while(1)
{
printf("Running...\n");
sleep(3);
}
return 0;
}

OUTPUT:

```
vaibhavi@VYSRI:~/OSSP$ nano signal.c
vaibhavi@VYSRI:~/OSSP$ gcc signal.c -o signal
vaibhavi@VYSRI:~/OSSP$ ./signal
Running...
Running...
Running...
Running...
Running...
Running...
Running...
Running...
Running...
Running...
Running...
Running...
Running...
Running...
Running...
Running...
Running...
Running...
Running...
Running...
Running...
Running...
Running...
```





PRACTICAL 6b:

1.Develop a program that executes the equivalent of the shell command: ls -l | grep ".c" using fork(), pipe(), dup2(), and exec() system calls.

```
vaibhavi@VYSRI:~/OSSP$ mkdir -p ~/OSSP/Practical6
vaibhavi@VYSRI:~/OSSP$ cd ~/OSSP/Practical6
vaibhavi@VYSRI:~/OSSP/Practical6$ nano pipe.c
vaibhavi@VYSRI:~/OSSP/Practical6$
```

vaibhavi@VYSRI: ~/OSSP/Pra

GNU nano 8.7.1 pipe.c *

```c
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main()
{
    int fd[2];

    pipe(fd);

    if(fork() == 0)
    {
        dup2(fd[1], STDOUT_FILENO);
        close(fd[0]);
        close(fd[1]);

        execlp("ls", "ls", "-l", NULL);
    }

    if(fork() == 0)
    {
        dup2(fd[0], STDIN_FILENO);
        close(fd[0]);
        close(fd[1]);

        execlp("grep", "grep", ".c", NULL);
    }

    close(fd[0]);
    close(fd[1]);

    wait(NULL);
    wait(NULL);

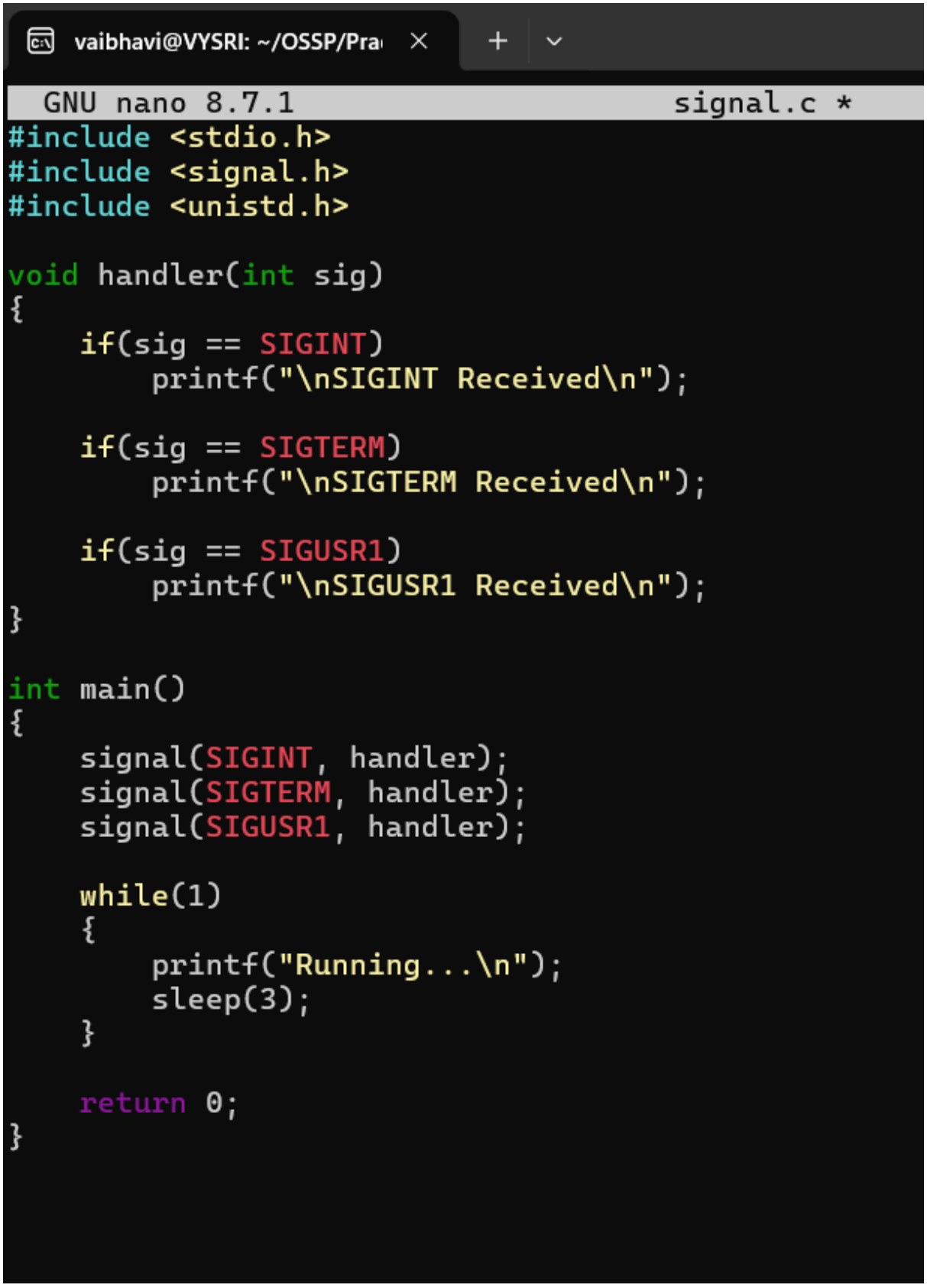
    return 0;
}
```

OUTPUT:

```
vaibhavi@VYSRI:~/OSSP/Practical6$ gcc pipe.c -o pipe
vaibhavi@VYSRI:~/OSSP/Practical6$ touch sample.c
touch program.c
touch notes.txt
vaibhavi@VYSRI:~/OSSP/Practical6$ ./pipe
-rw-r--r-- 1 vaibhavi vaibhavi     0 Oct 13 03:50 notes.txt
-rwxr-xr-x 1 vaibhavi vaibhavi 16208 Oct 13 03:50 pipe
-rw-r--r-- 1 vaibhavi vaibhavi   503 Oct 13 03:49 pipe.c
-rw-r--r-- 1 vaibhavi vaibhavi     0 Oct 13 03:50 program.c
-rw-r--r-- 1 vaibhavi vaibhavi     0 Oct 13 03:50 sample.c
vaibhavi@VYSRI:~/OSSP/Practical6$
```

2.Create a POSIX signal handling program that captures SIGINT, SIGTERM, and SIGUSR1. Demonstrate asynchronous event handling and explain the role of signal handlers.

```
vaibhavi@VYSRI:~/OSSP/Practical6$ cd ~/OSSP/Practical6
vaibhavi@VYSRI:~/OSSP/Practical6$ nano signal.c
```


vaibhavi@VYSRI: ~/OSSP/Pra
GNU nano 8.7.1 signal.c *
#include <stdio.h>
#include <signal.h>
#include <unistd.h>
void handler(int sig)
{
if(sig == SIGINT)
printf("\nSIGINT Received\n");
if(sig == SIGTERM)
printf("\nSIGTERM Received\n");
if(sig == SIGUSR1)
printf("\nSIGUSR1 Received\n");
}
int main()
{
signal(SIGINT, handler);
signal(SIGTERM, handler);
signal(SIGUSR1, handler);
while(1)
{
printf("Running...\n");
sleep(3);
}
return 0;
}

OUTPUT:

```
vaibhavi@VYSRI:~/OSSP/Practical6$ gcc signal.c -o signal
vaibhavi@VYSRI:~/OSSP/Practical6$ ./signal
Running...
Running...
Running...
Running...
Running...
Running...
Running...
Running...
Running...
Running...
Running...
Running...
Running...

SIGUSR1 Received
Running...
Running...
Running...
Running...
Running...

SIGTERM Received
Running...
Running...
Running...
Running...
Running...
Killed                  ./signal
vaibhavi@VYSRI:~/OSSP/Practical6$
```

